Spreeland Logistics Sync: Protocol Architecture Design

This document details the architectural design for orchestrating a resilient logistics network in the Spreeland region, addressing the requirements of **Problem Set 12**. We leverage the **Agentic Protocol Stack** to ensure secure, scalable, and dynamic interactions across databases, sub-agents, financial clearing, and the user interface.

🏛️ Protocol Architecture Mapping

The Spreeland logistics system relies on specialized layers within the Agentic Protocol Stack. Below is the mapping of each system interaction to its corresponding protocol:

System Interaction	Protocol Used	Primary Function
1. Infrastructure Discovery (Database status)	**MCP** (Model Context Protocol)	Connects the Dispatch Agent directly to the City's PostgreSQL database tools.
2. Expert Consultation (Weather predictions)	**A2A** (Agent-to-Agent Protocol)	Facilitates discovery, routing, and tasks delegation to external agent experts.
3. Secure Fulfillment (Wholesale gherkin purchase)	**UCP** & **AP2** (Commerce & Payments)	Standardizes the checkout flow (UCP) and provides cryptographically signed payment authorization (AP2).
4. Dynamic Visualization (Real-time dashboard)	**AG-UI** & **A2UI** (Streaming & Layout)	Streams live updates (AG-UI) and renders declarative JSON UI elements safely (A2UI).

🔍 Detailed Interaction Analysis

1. Infrastructure Discovery: Checking Bridge Status via MCP

To fetch real-time bridge conditions from the City's PostgreSQL database, we implement a **Model Context Protocol (MCP)** integration.

- **Mechanism**: An MCP server (e.g., `@spreeland/bridge-mcp-server`) runs as a database wrapper on the City's host. It registers specific database querying capabilities as schema-defined *tools* (e.g., `check_bridge_status`, `get_active_maintenance`).

- **Execution Flow**: The `Spreeland_Dispatcher` agent initiates an MCP connection as a client (using `StdioConnectionParams` to run the npx script). Through the MCP handshake, the agent dynamically discovers these tools, allowing it to invoke SQL-backed status queries without writing raw SQL or embedding proprietary database drivers inside the cognitive core.
- **Benefits**: Decouples the agent logic from the database schema, ensures the database credentials are kept isolated on the server side, and allows the City to enforce read-only access controls.

---

2. Expert Consultation: Querying the Weather-Predictor Agent via A2A

Logistics routes depend on fluctuating water levels. Rather than building forecasting logic from scratch, the dispatcher consults an external specialized weather agent.

- **Mechanism**: We utilize the **A2A (Agent-to-Agent)** protocol. It provides a standardized service discovery and messaging schema for cross-framework agent communication.
- **Execution Flow**: The `Spreeland_Dispatcher` looks up the `weather_predictor` agent in the workspace registry. Under the ADK framework, this agent is registered as a sub-agent. When the dispatcher's LLM determines it needs weather forecasts, it calls the `weather_predictor` tool. The A2A layer forwards the context, executes the sub-agent's reasoning loop, and streams back the structured output.
- **Benefits**: Enables multi-team collaboration where agents are black boxes built on different LLMs or frameworks (e.g., LangGraph vs. CrewAI) but communicate via unified input-output specifications.

---

3. Secure Fulfillment: 2 Tons Gherkin Purchase via UCP & AP2

Executing a wholesale purchase on behalf of the owner requires both a commercial negotiation protocol and a hard security standard for payment authorization.

- **Mechanism**: We split this concern into a commerce layer (**UCP**) and a secure payment layer (**AP2**).
  - **Universal Commerce Protocol (UCP)**: Used to manage the merchant shopping loop. The Dispatch Agent talks to the farmer cooperative's UCP server to select the 2 tons of gherkins, add them to a transaction cart, verify stock levels, and generate a pre-checkout invoice.
  - **Agent Payments Protocol (AP2)**: Handles the authorization constraint. The AP2 layer blocks execution and prompts the user/owner for explicit approval. Once authorized (e.g., using a biometric FIDO key, passkey, or cryptographic signature), the AP2 protocol signs the payment request. The transaction is executed, generating a cryptographically signed transaction hash (receipt) for auditing.
- **Benefits**: Mitigates the risk of rogue AI spending by establishing a hard cryptographic boundary of human authorization (chain of trust) while automating the tedious checkout steps.

---

4. Dynamic Visualization: Live Dashboard Rendering via AG-UI & A2UI

To present a live delivery dashboard without writing custom React, Flutter, or HTML code, we split the delivery between communication and rendering protocols.

- **Mechanism**:
  - **AG-UI (Agent-User Interaction)**: A lightweight, event-driven streaming protocol that maintains a persistent bi-directional connection (e.g., SSE or Websockets) between the dispatcher runner and the web interface. It pushes partial reasoning, status logs, and UI update events in real-time.
  - **A2UI (Agent-to-User Interface)**: The dispatcher agent emits declarative UI update payloads. Rather than raw HTML or JS (which present severe XSS vulnerabilities), the agent outputs a structured JSON document representing components registered in a client-side component catalog (e.g., `Card`, `Column`, `Row`, `Text`, `Button`).
- **Execution Flow**:
  1. The agent decides to update the delivery status.
  2. It generates an A2UI `updateComponents` JSON payload containing the layout (see `a2ui_schema.json`).
  3. The AG-UI stream delivers this JSON payload to the user's browser.
  4. The client's pre-built A2UI renderer parses the JSON and instantiates native, safe visual cards for the user.
- **Benefits**: Total separation of agent concerns from the frontend layout, complete safety against XSS attacks, and native mobile/web rendering without redeploying code.